

## 2. Design Philosophy

This chapter is arguably the most important part of this document.

Many AI coding assistants tend to gradually redesign interfaces while implementing features. Small inconsistencies accumulate over time until the application loses its identity.

The purpose of this chapter is to prevent that.

Every UI decision should follow these principles.

### Overall Design Direction

The project follows a modern minimalistic design language inspired by:

- Linear
- Notion
- Stripe
- Raycast
- Vercel
- Apple Human Interface Guidelines
- Swiss graphic design

The goal is clarity, not decoration.

The interface should feel professional, calm and trustworthy.

When a new user opens Peregri, they should immediately feel that the platform is reliable enough to trust strangers with expensive deliveries.

This is especially important because the platform deals with money and international shipping.

## Design Keywords

Every screen should communicate: Minimal. Calm. Precise. Elegant. Lightweight. Trustworthy. Professional. International.

Never: Playful. Gaming. Crypto. Neon. Over-designed. Material Design 2014.

## Visual Hierarchy

The interface should rely on spacing rather than borders.

Whitespace is an important design element. Content should breathe.

Information should be grouped naturally instead of enclosed inside heavy containers.

Users should immediately understand: primary action, secondary action, supporting information -- without excessive colors.

## Simplicity over Decoration

Every decorative element should justify its existence.

Avoid: unnecessary gradients, glowing effects, oversized shadows, animated backgrounds, decorative illustrations, floating shapes, excessive icons.

Minimalism is not the absence of design. It is the removal of everything unnecessary.

## Color Philosophy

Colors communicate function. Not decoration.

The interface intentionally uses a restrained, near-black neutral palette with a single blue accent, rather than a broad multi-hue scale. Accent is reserved for important interactions: primary buttons, selected states, important links, focused inputs.

Avoid introducing random colors for individual components.

## CSS Variables

The project's actual design tokens (defined once in `index.css`) are:

```
--bg          #09090b  base page background
--surface     #0a0a0c  card / panel background
--surface-raised #111113  hover / raised surface
--border      #27272a
--accent      #2563eb  primary blue accent
--accent-dim   #1d4ed8  pressed / focus-ring variant of accent
--text        #e4e4e7  primary text
--text-muted   #71717a  secondary text
--text-faint   #3f3f46  tertiary / disabled text
--success      #22c55e
--destructive  #ef4444
```

A spacing scale is also centralized as CSS variables: `--space-1` through `--space-8` (4 / 8 / 16 / 24 / 40 / 64 / 96 / 128px).

Components should never invent new raw colors. Instead they consume these tokens.

Good: `color: var(--text-muted);`

Bad: `text-zinc-500` (a value that happens to look similar today, but isn't wired to the design system and will drift the next time the palette changes).

## Tailwind Philosophy

Tailwind is used as an implementation tool. It is NOT the design system.

The tokens above are re-exposed as Tailwind theme colors (via `@theme` inline in `index.css`, plus shadcn's semantic layer: `--background`, `--foreground`, `--card`, `--primary`, `--secondary`, `--muted`, `--destructive`, etc., all aliased back to the Peregri tokens). This means components should reach for the semantic Tailwind utility classes that map to those variables, not for raw palette classes.

Correct:

`bg-primary / text-primary-foreground`

`bg-secondary / text-secondary-foreground`

`bg-surface-raised / text-text-muted`

`border-border`

Incorrect:

`bg-black`

`bg-slate-900`

`text-gray-500`

Known exception: the marketing Landing page (`src/pages/Landing.tsx`) currently uses raw Tailwind palette classes (`zinc-900/zinc-800`, `blue-600`) directly instead of the semantic tokens above. This predates the current token system and is a known inconsistency, not an endorsed pattern -- new marketing sections should use the tokens like the rest of the app, and this page is a candidate for a future cleanup pass rather than a model to copy from.

## shadcn/ui Philosophy

shadcn/ui is used as a component foundation. Its default appearance should not dictate the project's style.

Every component should be adjusted to match the Peregri design language. If a default shadcn component conflicts with the design philosophy, customization is preferred over accepting the default.

## Border Radius

Rounded corners should remain subtle. The application intentionally avoids exaggerated rounding.

In practice, the codebase sticks to two radii: rounded-sm for cards, inputs and containers, and rounded-full for pills, avatars and buttons (the button component itself is fully rounded, which reads as a deliberate brand shape here rather than a "playful" pill -- the rule to avoid is decorative rounding on cards and panels).

Avoid: rounded-2xl, rounded-3xl, pill-shaped cards. Cards should feel clean rather than playful.

## Shadows

Shadows exist only to separate elevation levels. Avoid heavy shadows. The interface should not resemble floating glass objects. Light shadows. Soft shadows. No dramatic effects.

## Typography

Typography carries most of the visual hierarchy. Instead of making everything colorful: increase font weight, adjust spacing, modify size. Good typography reduces the need for decoration.

The actual typefaces loaded by the app are DM Sans (UI text and headings, weights 300/400/500), DM Serif Display (available for display-style headings), and JetBrains Mono (monospace, used for numeric/code-like details). Do not introduce additional font families without updating index.html and index.css together.

## Icons

Icons should support labels. Not replace them. Avoid interfaces that communicate entirely through icons. Most important actions should remain understandable even if icons disappear.

## Buttons

Buttons should have clear hierarchy. The actual component (`components/ui/button.tsx`) exposes: default (primary), secondary, outline, ghost, destructive (danger), and link.

Only one primary (default) action should exist in each logical section. Multiple competing primary buttons reduce clarity.

## Forms

Forms should feel effortless. Generous spacing. Clear labels. Helpful validation. Minimal distractions. Users should never wonder "What does this field mean?"

## Mobile Philosophy

The application is mobile-first. Most users will discover listings while traveling. Every screen should feel natural on a phone before being optimized for desktop.

Desktop layouts should simply expand existing mobile layouts rather than becoming completely different interfaces.

## Animations

Animations should support understanding. Never entertainment.

Acceptable: fade, small scale, smooth transitions.

Avoid: bounce, elastic movement, overshooting, large rotations, attention-grabbing motion.

Users should barely notice animations. But they should feel the interface is polished.

## Empty States

Empty states should encourage action. Not punish users.

Instead of: "No listings found."

Prefer: "No travelers match these filters yet. Try expanding your search or creating your own request."

Always provide a next step. Note: this is a target standard, not yet applied consistently everywhere -- e.g. Browse currently shows the shorter "No listings match your filters." Bring copy in line with this guideline as those screens are touched.

## Error Messages

Errors should explain: what happened, why, what to do next.

Avoid generic: "Something went wrong."

Instead: "We couldn't load available trips. Please check your connection and try again."

Note: this is also a target standard the codebase doesn't fully meet yet (a couple of flows still surface a bare "Something went wrong."); tighten wording toward this guideline when touching those flows rather than treating the current copy as the reference.

## Accessibility

Accessibility is considered from the beginning.

Requirements: proper semantic HTML, keyboard navigation, visible focus states, sufficient contrast, meaningful labels.

The goal is not merely passing automated tests. The interface should genuinely be usable.

## Consistency

Consistency is more valuable than novelty.

When implementing a new feature: prefer existing layouts, reuse existing components, reuse spacing, reuse typography, reuse colors.

The user should feel every page belongs to the same application.

## AI Design Rules

Every AI agent contributing to the frontend must follow these rules:

- Never redesign existing pages unless explicitly instructed.
- Never replace the design tokens (CSS variables / semantic Tailwind classes) with raw Tailwind palette colors.
- Never introduce a second design language.
- Never increase border radius without approval.
- Prefer reusing components over creating similar ones.
- Preserve spacing consistency.
- Preserve typography hierarchy.
- Preserve minimalist philosophy.
- Match existing pages before inventing new styles.

The objective is for a user to never guess which page was built first or by whom. Every screen should appear as if it was designed by the same person on the same day.